Being encoding-aware, these functions are safe to use on strings that would otherwise break…

```
// raw byte data for UTF-16BE string " utf-16"
$utf16 = "\x00\x20\x00\x75\x00\x74\x00\x66\x00\x2d\x00\x31\x00\x36";
print bin2hex($utf16) . PHP_EOL;
print bin2hex(ltrim($utf16)) . PHP_EOL;
print bin2hex(mb_ltrim($utf16, encoding: 'UTF-16BE'));
```

```
Output:
00200075007400660002d00310036
7500740066002d00310036
007500740066002d00310036
```

Notice that the output of `ltrim()` has removed half of the UTF-16 encoding for the letter u:*0x00 0x75* has been truncated to *0x75*.

str_pad(), mb_strpad

The corollary to trimming strings is padding them: adding characters to either or both ends. `str_pad()` – in the grand tradition of consistency among PHP's core strings functions – does not have a left or right form. Directionality is controlled by the function argument.

```
$str1 = "First";
$str2 = "Second";
print str_pad($str1, 6) . $str2 . PHP_EOL;
```

```
Output:
First Second
```

Here we've asked `str_pad()` to pad the input `$str1` so it is six bytes (ASCII characters) wide. The default padding direction is right (onto the end of the string), and the default byte is *0x20* (a single space). The input string is five bytes/characters wide, so just a single padding character is added, as we can see in the output. Additional arguments allow control of direction and padding bytes:

```
print str_pad($str1, 11, ',.,', STR_PAD_BOTH) . $str2 . PHP_EOL;
```

```
Output:
,.,First,.,Second
```